Entrega 2: guía de ejecución y revisión

Compiscript produce código de tres direcciones tipado y ubicado, y un reporte estático de símbolos extendidos, mediante un único núcleo de compilador. La traducción (lowering) cubre todos los tipos de construcción válidos del AST, incluidas las adiciones de valores/efectos y de valores desconocidos protegidos/try-catch. Empiece aquí, desde la raíz del repositorio. Es un compilador, no un servicio de ejecución de código fuente ni de TAC. El informe separa los contratos implementados de la calificación, la autoría y las comprobaciones manuales para la presentación.

Mapa de la documentación

Necesidad	Referencia
Resultado académico, requisitos del PDF, contribuciones, brechas resueltas y comprobaciones humanas	Informe técnico
Flujo, responsabilidades y decisiones prácticas de diseño	Arquitectura
Sintaxis completa de las instrucciones, ejemplos de traducción y supuestos del backend	Referencia TAC
Batería ejecutable de casos exitosos y fallidos, y límites de la evidencia	Aceptación
Cargar/editar/compilar/analizar, paneles y guardas HTTP	Guía del IDE
Enunciado completo de cuatro páginas, límites de página originales y rúbrica	Instrucciones canónicas

Paquete de PDF descargables

- Informe técnico
- Arquitectura
- Guía de ejecución y revisión
- Referencia TAC detallada, ejemplos de traducción y supuestos

Los PDF son exportaciones legibles de las fuentes Markdown editables anteriores, no evidencia nueva de implementación ni de certificación en navegador. Mantenga juntos los cuatro PDF para que funcionen los enlaces del paquete; las demás referencias abren URL de rutas de una rama del repositorio, no permalinks inmutables. Consulte el índice de entregables y la política de versiones. Las instrucciones canónicas conservan el enunciado completo; el `Proyecto_2__Compiladores.pdf` original se mantiene local e ignorado. Regenere el paquete después de cambiar sus fuentes Markdown y versione ambos juntos.

Requisitos previos

- Go 1.26.1 o posterior, según go.mod; la verificación local actual usó Go 1.27.2 en darwin/arm64.
- Dependencias de módulos Go existentes: runtime de ANTLR para Go v4.13.1 y la versión indirecta fijada de `golang.org/x/exp` en `go.mod`. No se requiere ningún cambio de dependencias.
- Node.js para `node --test`; versión observada 24.21.0. Estas pruebas integradas no requieren `npm install`.
- Java solo para regenerar ANTLR; versión observada OpenJDK 17.0.20.1. El jar del generador es ANTLR 4.13.2, almacenado en `tools/antlr/antlr-4.13.2-complete.jar`. Las versiones del generador y del runtime de Go son distintas. Una copia nueva sin conexión necesita el mismo jar y los mismos módulos en caché; ni el Markdown ni Git por sí solos los garantizan.
- Un navegador para el IDE local. Los recursos CDN existentes de CodeMirror/Viz.js hacen que la apariencia en el navegador y el funcionamiento sin conexión dependan de su disponibilidad. No se garantiza el funcionamiento del navegador sin conexión.

Generar código

```
go run ./cmd/compiscript-tac -h
go run ./cmd/compiscript-tac testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript testdata/compiscript/tac/acceptance.cps
```

El TAC numérico termina con una escritura en `s1.x:float`; representa `x = 3.5`, pero no imprime ese valor en tiempo de ejecución. El JSON de una compilación exitosa contiene `tac`, `diagnostics` vacío, `scopes` semánticos

y `codegen` opcional con unidad `abstract-cell`. El último comando es la interfaz Analyze heredada: AST, diagnósticos y ámbitos, sin TAC ni las adiciones de codegen.

Los dos nuevos archivos de prueba generan instrucciones de print, guardas y manejadores. Solo el oráculo acotado de pruebas las ejecuta: `values-effects.cps` produce `3\n1\nnull\n` con `x=2`, `y=3`, `count=1`; `dynamic-exceptions.cps` produce `3\ndivision by zero\n11\n` con `result=3`, `effects=11`. Son expectativas de las pruebas, no la salida estándar de la CLI. Los valores desconocidos pueden compilar con éxito y después encontrar un fallo explícito en tiempo de ejecución; compilar no certifica que la ejecución sea segura. Consulte los contratos de valores y manejadores.

Entrada fallida y código de salida exacto

```
go run ./cmd/compiscript-tac testdata/compiscript/acceptance/operators_invalid.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/acceptance/operators_invalid.cps
```

Este archivo de prueba produce seis diagnósticos `SEM_OPERATOR`. El rechazo en texto deja stdout vacío y muestra los errores ubicados en stderr. El rechazo en JSON deja el reporte en stdout, stderr vacío, `tac: null` y sin `codegen`. Un binario TAC compilado termina con **0** ante éxito o ayuda, con **1** ante diagnósticos o fallos de E/S, y con **2** ante errores de argumentos, formato o extensión. Las opciones preceden a la ruta. El Analyze heredado sigue terminando con **0** ante diagnósticos del lenguaje. `go run` envuelve los códigos de salida distintos de cero del programa; para comprobar el código exacto, use las pruebas existentes de la CLI construida, no el código del envoltorio.

Usar el IDE

```
go run ./cmd/ide
```

Abra `http://localhost:8080`. La dirección `:8080` y la raíz relativa de archivos `web` están definidas en cmd/ide/main.go, por lo que debe iniciarse desde la raíz del repositorio. El servidor escucha en la dirección comodín, no exclusivamente en loopback; úselo solo en un entorno local de confianza. Detenga el servidor en primer plano con Ctrl+C.

Seleccione Compiscript, cargue un archivo `.cps` o edite el código, y luego elija **Compile .cps**. El TAC, los diagnósticos, los entornos y los símbolos extendidos son vistas separadas. **Analyze .cps (AST)** conserva el flujo de análisis anterior. Consulte los detalles del IDE sobre reinicio, respuestas obsoletas y preservación de YAPar.

Con un servidor ya iniciado localmente, esta solicitud ilustra el contrato público (no es una solicitud de ejecución):

```
curl -sS http://localhost:8080/api/compiscript/compile -H 'Content-Type: application/json' --data '{"so
```

Un HTTP 200 puede contener diagnósticos del lenguaje; no significa que la fuente haya compilado con éxito. Los fallos de la solicitud usan 400/413/415/405 por separado. Durante la comprobación de la documentación no se inició ningún servidor ni solicitud curl; httptest prueba la frontera del manejador sin dejar un servidor activo.

Ejecutar la batería de forma segura

Use los comandos de aceptación focalizados. Ejercitan el éxito, el rechazo atómico, los códigos de salida de la CLI, la paridad HTTP y la ejecución acotada del TAC emitido. Un `go test ./...` amplio puede regenerar archivos ANTLR ignorados; ejecute la generación, compilación y vet completos solo en una instantánea aislada del código. Las comprobaciones sin conexión requieren `GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local`, un `GOMODCACHE` existente y HOME/TMPDIR/GOCACHE externos. No infiera la capacidad de instalar sin conexión a partir de una ejecución que ya tenía caché.

Antes de la presentación

Revise el estado actual y las comprobaciones humanas, inspeccione manualmente el navegador y verifique el acceso a GitHub y la autoría individual real. La brecha original de construcciones aceptadas quedó resuelta con T9/T10; esta actualización no establece ninguna brecha normativa concreta adicional de implementación. Eso no prueba todos los programas ni las condiciones futuras de la presentación. Los 25/65/10 puntos del PDF son pesos de calificación, no porcentajes de avance.